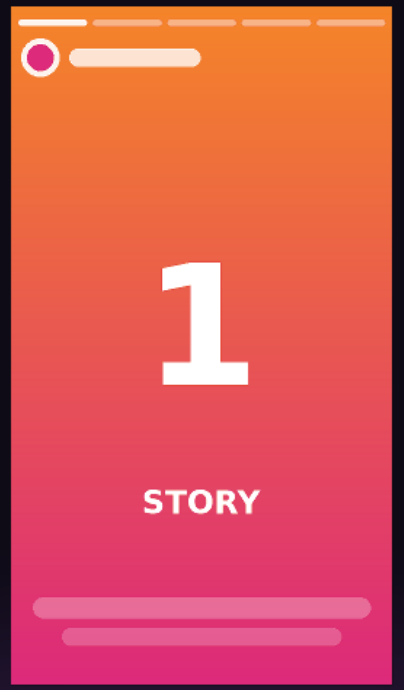
THE INSTAGRAM

Stories Cube

Zero → Master

Part 1: CSS 3D from a single square.
Part 2: the full Stories engine — pivot, gestures, infinite recycling.



animation on this slide: rendered with the exact math in this deck

`perspective · rotateY · matrices · virtualization`

PART 1

3D from zero

No Instagram yet. Just a humble square, the three axes, and the one formula that turns flat pixels into depth.

- the third axis

`rotateX / rotateY / rotateZ`

`perspective & its value`

`scale = d / (d − z)`

`perspective-origin`

`transform order`

Why bother with theory?

Because in Part 2 every Instagram decision — the 1000px, the −w/2, the 45° threshold — falls out of these six ideas with zero magic numbers.

Meet the third axis

Animate translateZ on a square and watch both panels. Same code — one of them is missing a viewer.

WITHOUT perspective `z = +0px`



translateZ... does nothing?!

`perspective: 800px`



closer = bigger. There it is.

x
left ↔ right

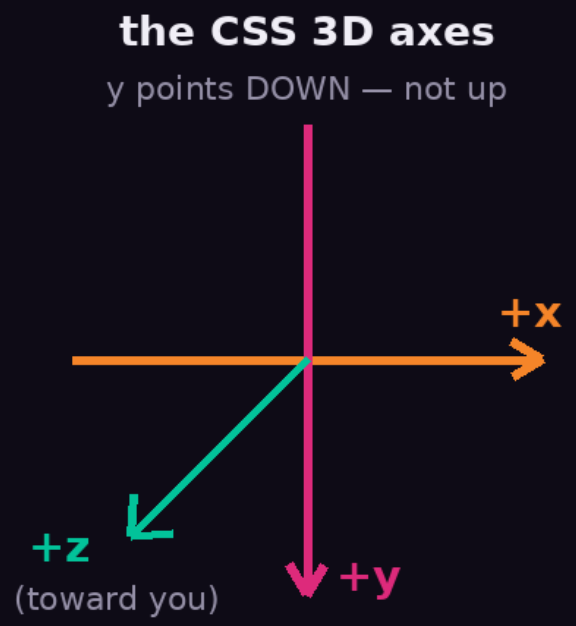
y
up ↔ down

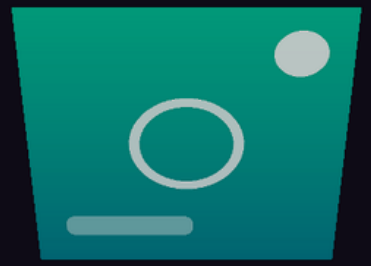
z
toward ↔ away from YOU. Invisible on screen until something says where you stand.

transform: translateZ(z) moves the square along the axis pointing out of the screen — the axis our cube will rotate stories around.

The three axes — and the one gotcha

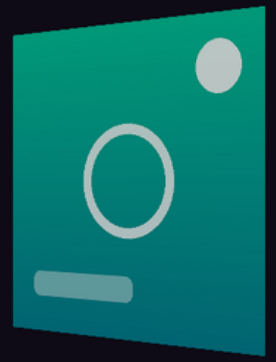
Same square, three rotations. Only rotateY will build our cube later.





`rotateX(…)`

tips away — top/bottom depth



`rotateY(…)`

turns — left/right depth



`rotateZ(…)`

flat spin — no depth at all

translateX slides sideways · **translateY** slides down (y is inverted vs. math class!) · **translateZ** moves toward/away from you — invisible until perspective exists.

Foundations: giving the browser a third axis

Three properties turn a flat document into a 3D scene.

`perspective: 1000px`	Placed on the PARENT. Defines d — the distance from your eye to the screen plane. Smaller d = fisheye drama; larger d = flat telephoto.
`transform-style: preserve-3d`	Children keep their own z-positions instead of being flattened into the parent's plane. Without this, there is no cube — only a picture of one.
`backface-visibility: hidden`	When a face rotates past 90°, its mirrored back would show through. Hiding backfaces keeps the illusion airtight.

Coordinate system: x → right, y → down, **z → out of the screen toward you.** translateZ(+) moves closer, translateZ(−) moves away.

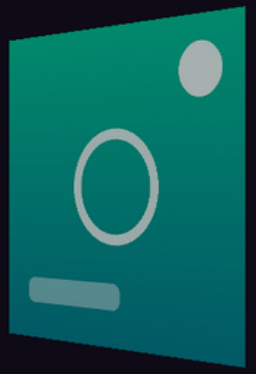
What perspective actually is: where YOU stand

Same square, same 45° turn — four different viewers. Perspective is a camera position, not a style.



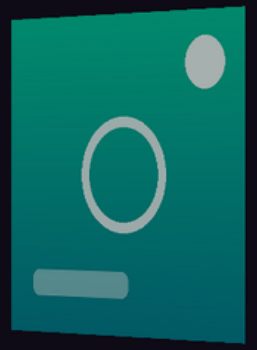
`perspective: 400px`

you stand close — dramatic



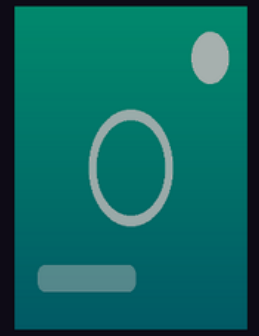
`perspective: 1000px`

balanced (Stories uses this)



`perspective: 2500px`

far away — subtle



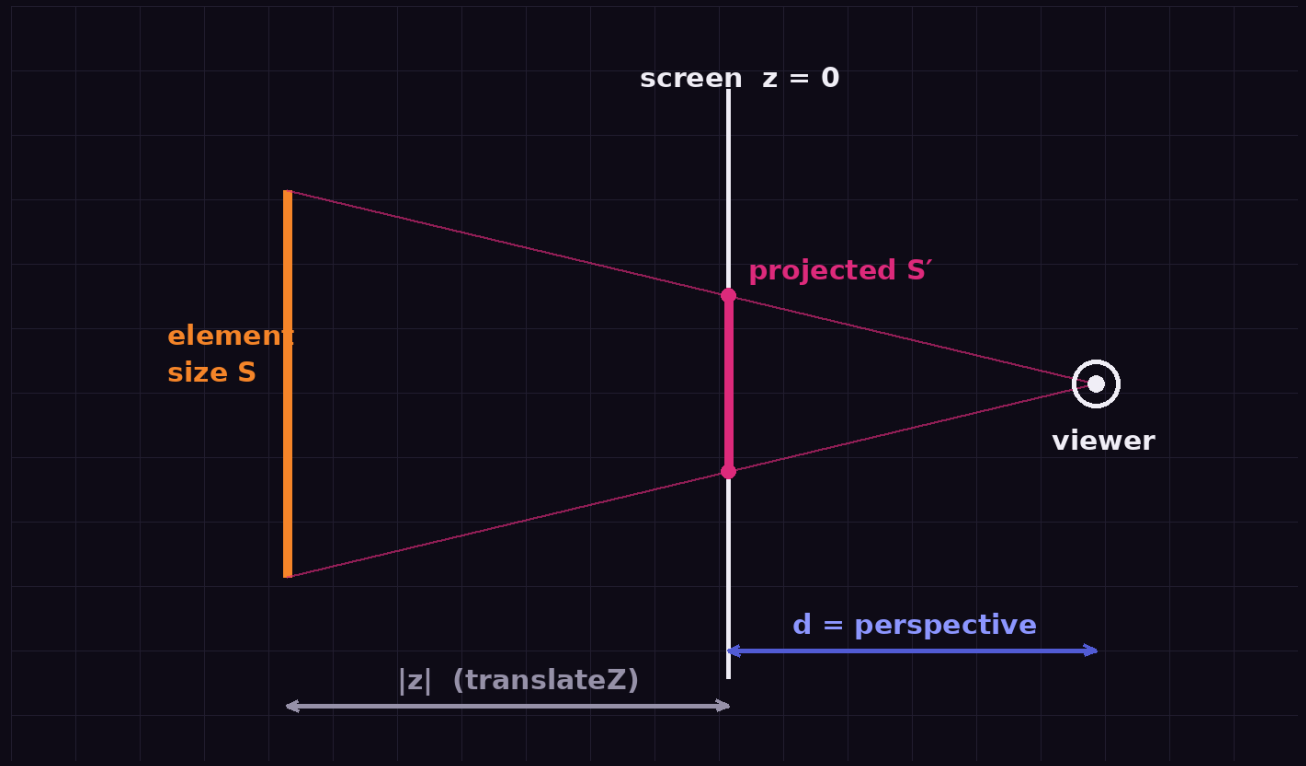
`no perspective`

just looks narrower

One number = your distance from the screen.
Small value → you stand close → edges bulge dramatically. Large value → you stand far → depth flattens out. It is a camera position, not a style.

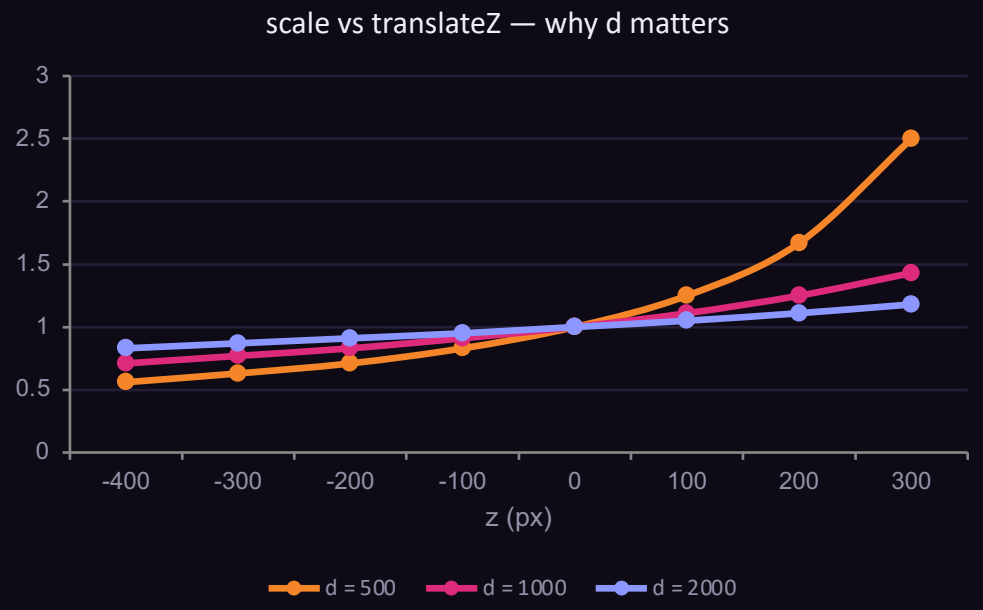
perspective: (property, on the parent) — ONE shared vanishing point for all children. This is what we will use.
perspective(): (function, in transform) — each element gets its own camera. Siblings would disagree about the vanishing point — multi-face shapes would tear apart.

The one formula behind every 3D effect



$$scale = d / (d - z)$$

Similar triangles: a point at depth z is drawn scaled by this factor around the perspective origin. That is the whole projection.



Stories will use perspective(1000px). When a face turns, one edge swings toward you (z > 0 → drawn bigger) and the other swings away (z < 0 → drawn smaller). This formula creates that asymmetry — and the asymmetry IS the 3D look.

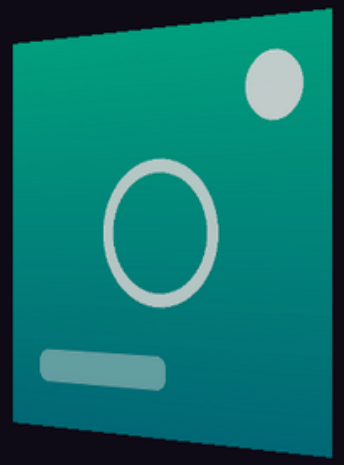
perspective-origin: where the viewer's eye sits

The formula scales everything AROUND this point — move it, and the vanishing point moves with it.



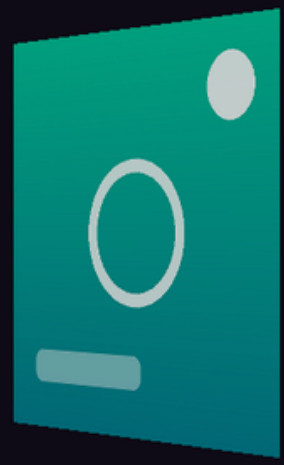
`perspective-origin: 0% 50%`

viewer stands to the LEFT



`perspective-origin: 50% 50%`

dead-center (default)



`perspective-origin: 100% 50%`

viewer stands to the RIGHT

Stories use the default 50% 50% — the phone fills the viewport, so your eye is assumed dead-center. Off-center origins are for scenes beside the viewer: carousels, shelves, parallax cards.

rotateY, as the GPU sees it

Every CSS transform is a 4×4 matrix. Rotation about Y mixes x and z.

RotateY(θ)

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$x' = x \cdot \cos\theta + z \cdot \sin\theta$$

$$z' = -x \cdot \sin\theta + z \cdot \cos\theta$$

Points gain depth (z′) as they rotate — and the perspective formula from the last slide turns that depth into on-screen scale.

Order matters — matrices do not commute

`rotateY(90°) translateZ(w/2)`
→ push along the ROTATED z: panel slides sideways into the cube's right wall. ✓ what we want

`translateZ(w/2) rotateY(90°)`
→ push toward the viewer first, then spin in place. ✗ no cube, just a panel floating in front

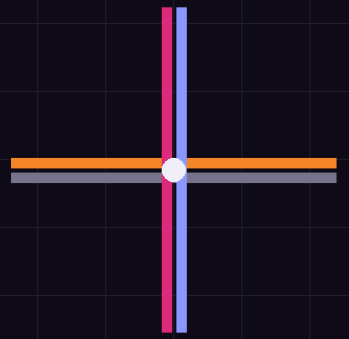
Read transform lists left → right: each function acts in the coordinate frame produced by the previous one.

Why translateZ? Spinning cards vs. building walls

rotateY alone turns each face in place, on the same hinge. The push along the ROTATED z is what makes a box.

rotateY only — NO translateZ

0° · 90° · 180° · 270°



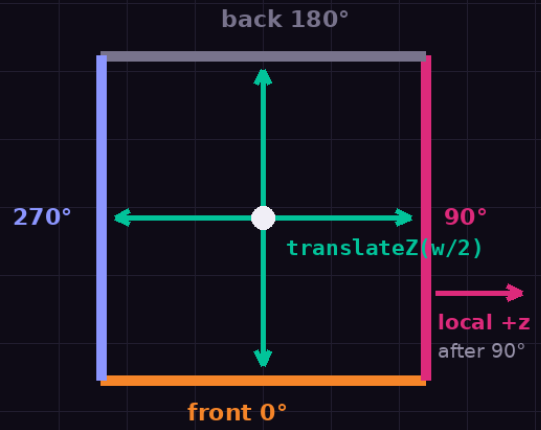
all four faces share ONE hinge

a pinwheel at a point — no box, no volume

from the front you'd see: one flat card

rotateY(…) translateZ(w/2)

each face pushed along its OWN rotated z



four walls, one shared volume: a cube

same rotations — one extra push each

Track ONE point — the face's center (0,0,0), w = 390:

rotateY(90°) translateZ(195px)
push happens along the rotated z (now pointing right)
→ center lands at (195, 0, 0): the cube's right wall ✓

translateZ(195px) rotateY(90°)
push happens first, toward you
→ center at (0, 0, 195): a card floating in front, spinning in place ✗

Each transform acts in the coordinate frame the previous one left behind. That is the whole rule. Our Stories deck skips the explicit translateZ because transform-origin: −w/2 builds the same rotate-around-a-shifted-point pair automatically.

Under the hood: everything is one 4×4 matrix

Every transform chain collapses into 16 numbers before rendering. Here is what each region does.

Anatomy of a transform matrix

$$\begin{bmatrix} a & b & c & tx \\ d & e & f & ty \\ g & h & i & tz \\ 0 & 0 & p & 1 \end{bmatrix}$$

3×3 block — rotate, scale, skew (linear maps)

last column — translate. Doesn't fit in 3×3: that is WHY the 4th coordinate exists

bottom row — perspective lives here (p = −1/d)

Where d/(d−z) actually comes from

```
perspective matrix · [x, y, z, 1]
        = [x, y, z, 1 − z/d]

then divide by the 4th component:
x_screen = x / (1 − z/d) = x · d/(d−z)
```

The formula from Part 1 is literally one matrix entry (−1/d) plus one division. No separate 'perspective system' exists.

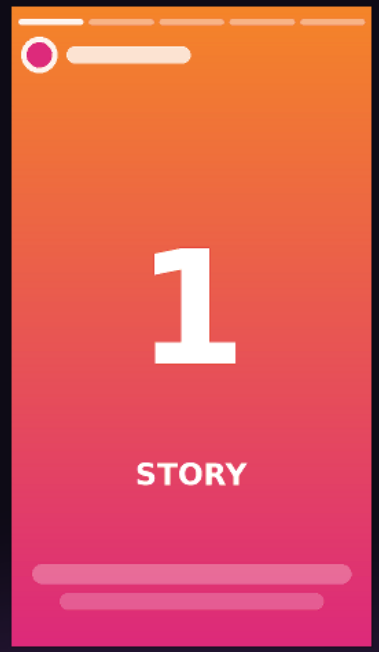
Homogeneous coordinates: every point carries a 4th component: (x, y, z) → [x, y, z, 1]. One matrix multiply then handles rotate AND translate AND perspective in a single operation — this is the trick all of 3D graphics runs on.

Chain = multiplication. rotateY(…) translateZ(…) scale(…) multiplies into ONE matrix — inspect it with getComputedStyle (matrix3d(…16 numbers)), or write it directly: transform: matrix3d(…). The GPU does one multiply per vertex: that is why animating transform is nearly free.

PART 2

The Stories cube

Everything from Part 1, aimed at one target:
this exact animation, at 60fps, for infinite stories.



- the pivot trick (-w/2)

assembling 3 faces

gesture → angle → snap

recycling to infinity

60fps rules

What we are actually building

Three ideas, stacked. Master each one and the effect is inevitable.

1

A 3D illusion

Two flat rectangles + one perspective projection = a convincing cube corner. Pure math, no 3D engine.

2

A gesture engine

Finger position maps linearly to a rotation angle; release maps velocity to a snap decision.

3

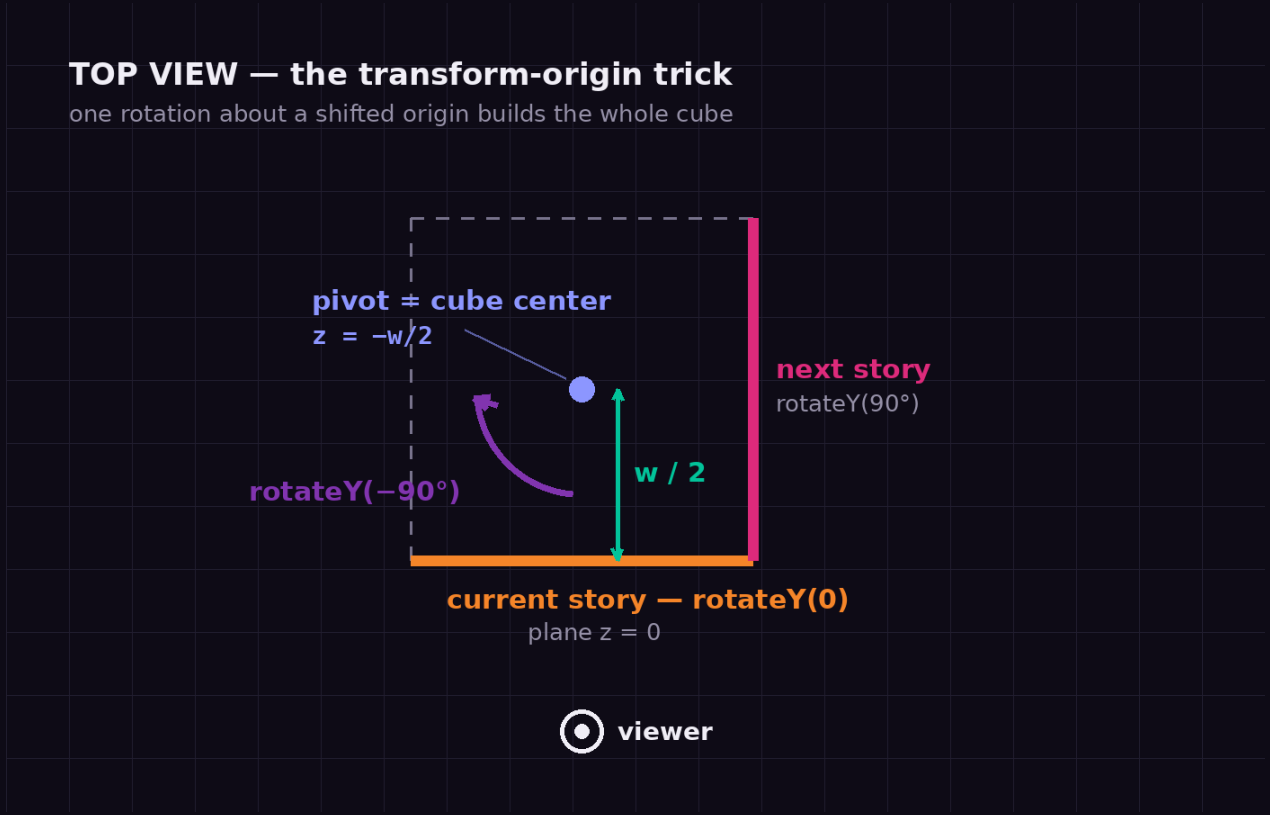
A window of 3

Only prev / current / next ever exist in the DOM. Panels are recycled — the cube never ends.

The roadmap: Part 1 gave you the math. Now: pivot trick → cube → gestures + snapping → infinite recycling → 60fps

Golden rule: the entire effect uses only transform. No layout, no paint — that is why it runs at 60fps on a phone.

The trick Instagram uses: move the pivot, not the panel



Instead of composing rotate + translateZ, shift the rotation origin to the cube's center:

```
.story-panel {
  transform-origin:
    50% 50% calc(var(--w) / -2);
}
```

Why it is the same math:

$$M = T(0,0,-w/2) \cdot R(\theta) \cdot T(0,0,+w/2)$$

Rotation about a shifted origin = translate the pivot to zero, rotate, translate back. The browser builds this matrix for you.

Payoff: every face uses the SAME origin and differs only by angle — current: rotateY(0), next: rotateY(90°), prev: rotateY(−90°). One variable drives the whole cube.

Step 1 — assemble the cube

Full-viewport faces, one shared pivot, one CSS variable for the spin.

```html
<div class="viewport">      <!-- perspective here -->
  <div class="cube">        <!-- rotates as one unit -->
    <section class="face prev">…</section>
    <section class="face current">…</section>
    <section class="face next">…</section>
  </div>
</div>
```

```css
.viewport { perspective: 1000px; overflow: hidden; }

.cube {
  transform-style: preserve-3d;
  transform-origin: 50% 50% calc(var(--w) / -2);
  transform: rotateY(var(--angle));  /* the ONLY mover */
}

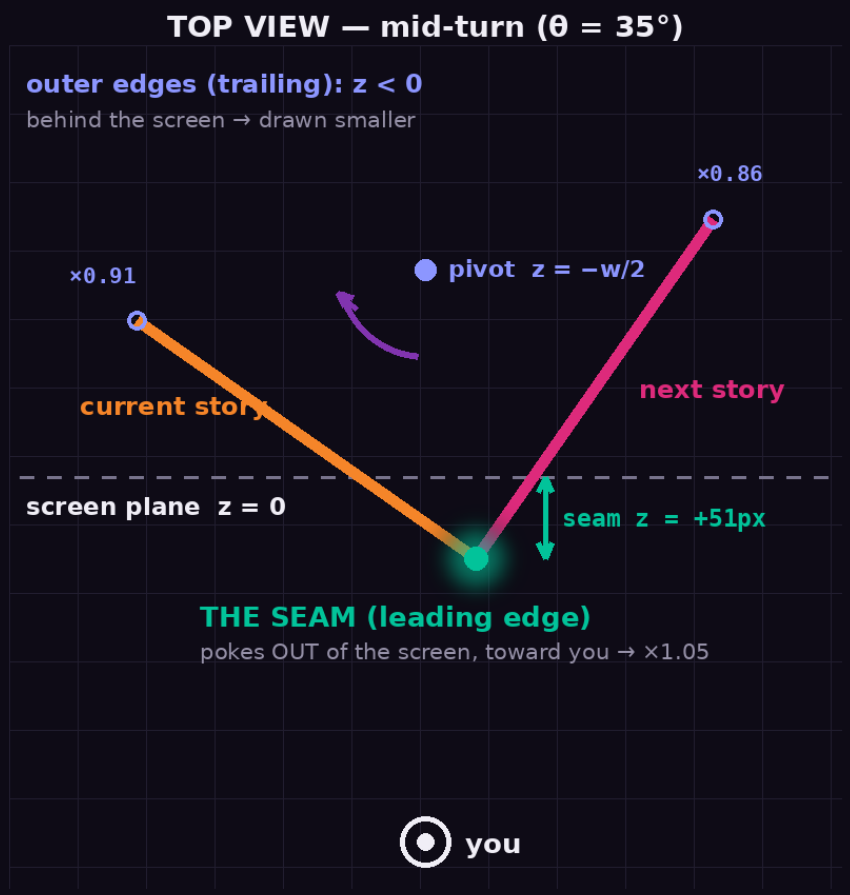
.face { position: absolute; inset: 0;
  transform-origin: 50% 50% calc(var(--w) / -2);
  backface-visibility: hidden; }
.prev    { transform: rotateY(-90deg); }
.current { transform: rotateY(0deg);   }
.next    { transform: rotateY(90deg);  }
```

Why −w/2 exactly?

The pivot must sit half a face-width behind the screen so adjacent faces meet edge-to-edge: the face spans x ∈ [−w/2, +w/2], and rotating its plane 90° about z = −w/2 lands it exactly on the neighbor's edge. Any other depth leaves a gap or an overlap at the corner.

Vocabulary of the turn: the seam vs. the outer edges

The front face lies ON your screen's glass. Turn the cube, and one corner pushes through it.





tall seam + shorter outer edges = the corner 'bends'

The seam (leading edge)

The vertical corner where the two stories meet. At rest it sits ON the screen plane (z = 0). Mid-turn it swings OUT in front of the glass, toward your face — it 'leads' the rotation.

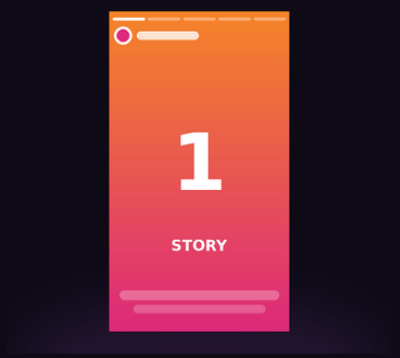
seam z

Simply the seam's depth: how many px it pokes out. 0 at rest → peak +53px at 45° → 0 again at 90°. This is the number under every filmstrip frame and in the demo HUD.

Outer edges (trailing)

The far-left and far-right sides of the two faces. They swing BEHIND the glass (z < 0), so perspective draws them shorter. Taller middle + shorter sides = a corner that visibly 'bends'.

The rotation, frame by frame



θ = 0°

seam z = +0px → ×1.00



θ = 13°

seam z = +26px → ×1.03



θ = 26°

seam z = +44px → ×1.05



θ = 39°

seam z = +53px → ×1.06



θ = 52°

seam z = +53px → ×1.06



θ = 65°

seam z = +43px → ×1.05



θ = 78°

seam z = +24px → ×1.02



θ = 90°

seam z = +0px → ×1.00

What to watch

The seam (previous slide) stays glued for the whole turn — the only thing both faces agree on, and pure −w/2 pivot math.

Depth peaks at 45°. Watch seam z under each frame: w/2·(sin θ + cos θ − 1), max ≈ 53px → only ×1.06 scale. The cube reads as 'deep' from a 6% bulge — perspective needs very little.

Perfect symmetry. θ and 90°−θ are mirror images, which is why a cancelled swipe rewinds without any visual pop.

Step 2 — finger position becomes an angle

The cube must feel attached to the finger. That is a linear map plus two guards.

$$\theta = 90° \times \Delta x / w$$

Drag a full screen-width → exactly one face (90°). Half a screen → 45°. Direct manipulation, zero easing while touching.

```
onMove(e) {
  const dx = e.clientX - startX;
  let deg = (dx / innerWidth) * 90;
  deg = clamp(deg, -90, 90);          // guard 1
  if (atFirstStory && deg > 0 ||
      atLastStory  && deg < 0)
    deg *= 0.25;                       // guard 2
  cube.style.setProperty('--angle', deg + 'deg');
}
```

The two guards, in plain words

Clamp to ±90°. One gesture moves at most one story. Past 90° you would see the back of the cube and the corner geometry breaks.

Rubber-band at the ends. At the first/last story, multiply the angle by ~0.25. The cube still responds — physical resistance instead of a dead wall — then springs back on release.

While dragging: no transition. transition: none during touch; the finger IS the animation curve. Easing only enters after release.

Step 3 — release: commit or cancel

Two signals decide; one eased transition finishes the story.

```
commit  =  |θ| > 45°
        OR |velocity| > 0.3 px/ms
```

Distance says intent, velocity says urgency. A short fast flick must advance even at only 20° — that flick-feel is what makes it native.

```
onEnd() {
  const commit = Math.abs(deg) > 45 ||
                 Math.abs(vx) > 0.3;
  const target = commit ? Math.sign(deg) * 90 : 0;
  const remain = Math.abs(target - deg) / 90;
  cube.style.transition =
    `transform ${250 * remain + 80}ms
     cubic-bezier(0.33, 1, 0.68, 1)`;  // easeOutCubic
  setAngle(target);
}
```

Why these numbers feel right

Duration ∝ remaining angle. Released at 80°? Only 10° remain — the snap must be quick. Released at 5°? Full travel, full duration. Constant-duration snaps feel laggy or jumpy.

Ease-out, never ease-in-out. The finger already provided the acceleration phase. The animation should only decelerate from the finger's velocity — matching curves is what 'buttery' means.

transitionend is your hook. When the snap finishes at ±90°, the recycling step on the next slide fires.

The complete engine — every line you need

Gesture + snap + recycle in ~30 lines. The working single-file demo ships with this deck.

```
const cube = document.querySelector('.cube');
const faces = [...cube.children];   // [prev, cur, next]
let index = 0, deg = 0, startX = 0,
    lastX = 0, lastT = 0, vx = 0, dragging = false;

const setAngle = v => { deg = v;
  cube.style.setProperty('--angle', v + 'deg'); };

addEventListener('pointerdown', e => {
  dragging = true; startX = lastX = e.clientX;
  lastT = e.timeStamp; vx = 0;
  cube.style.transition = 'none';       // finger = curve
});

addEventListener('pointermove', e => {
  if (!dragging) return;
  vx = (e.clientX - lastX) / (e.timeStamp - lastT || 1);
  lastX = e.clientX; lastT = e.timeStamp;
  let d = (e.clientX - startX) / innerWidth * 90;
  d = Math.max(-90, Math.min(90, d));
  if ((index === 0 && d < 0) ||
      (index === N - 1 && d > 0)) d *= 0.25;
  setAngle(d);
});
```

```
addEventListener('pointerup', e => {
  dragging = false;
  const commit = (Math.abs(deg) > 45 ||
                  Math.abs(vx) > 0.3) && canMove();
  const target = commit ? Math.sign(deg) * 90 : 0;
  const remain = Math.abs(target - deg) / 90;
  cube.style.transition = `transform
    ${250 * remain + 80}ms cubic-bezier(.33,1,.68,1)`;
  setAngle(target);
});

cube.addEventListener('transitionend', () => {
  if (deg === 0) return;
  index += deg < 0 ? 1 : -1;  // advance window
  cube.style.transition = 'none';
  setAngle(0);
  renderWindow();             // re-fill 3 faces
  void cube.offsetHeight;     // flush styles
});

function renderWindow() {
  [-1, 0, 1].forEach((o, i) =>
    fillFace(faces[i], stories[index + o]));
}
```

Step 4 — infinity from three panels

100 stories ≠ 100 DOM faces. A sliding window of 3 is recycled forever.



Why only 3?

During any gesture you can only ever see 2 faces — current plus one neighbor. Three panels (prev · current · next) cover both directions. More is wasted memory: stories are media-heavy, and phones kill greedy tabs.

The invariant

After every completed transition:
panels ← stories[i−1, i, i+1]
angle ← 0
The cube is always in the same 'ready' state, no matter how far you have traveled.

The invisible reset — the whole trick in one frame

Two different states that render identical pixels. Swap between them and nobody sees it.

State A (transition just ended)

```
cube: rotateY(−90°)
visible face: .next  →  holds story i+1
```

State B (after silent reset)

```
cube: rotateY(0°)
visible face: .current  →  holds story i+1
```

A ≡ B — pixel-for-pixel identical. rotateY(−90°) puts the next-face plane exactly where the current-face plane sits at rotateY(0°).

```
cube.addEventListener('transitionend', () => {
  if (angle === 0) return;              // was a cancel-snap
  index += Math.sign(-angle);           // advance the data window
  cube.style.transition = 'none';       // 1. kill animation
  setAngle(0);                          // 2. reset rotation
  renderWindow(index);                  // 3. re-assign 3 panels
  void cube.offsetHeight;               // 4. force style flush
  cube.style.transition = '';           // 5. re-arm for next swipe
});
```

Keeping it at 60 fps

The illusion dies at the first dropped frame. These rules keep every frame on the GPU.

Animate transform only

Rotation, translation, scale — all compositor work. Never animate left/top/width: those trigger layout on every frame.

will-change: transform

Promotes faces to their own GPU layers BEFORE the gesture starts. Add on touchstart, remove on idle — permanent layers eat memory.

Pause the hidden

Videos and timers on prev/next faces must pause. Two playing videos during a spin is how mid-range phones drop to 30fps.

Pointer events, passive scroll

Use pointer events + touch-action: pan-y so the browser never waits on JS to decide who owns the gesture.

Content async, chrome instant

Recycled panels show avatar + gradient instantly; the heavy media loads after. The cube must never wait for the network.

Measure on real phones

DevTools throttling lies about GPU. A $150 Android is the honest benchmark for a media-heavy cube.

Sanity check: open DevTools → Rendering → Paint flashing. During a swipe, the screen should stay completely un-flashed. Any green = you broke the golden rule.

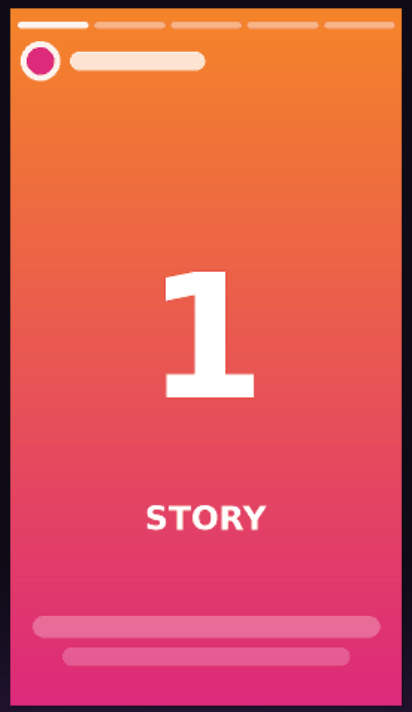
Master blueprint — the whole system on one slide



The three equations you now own

$\text{scale} = d/(d-z)$ the projection · $M = T \cdot R(\theta) \cdot T^{-1}$ the pivot · $\theta = 90° \cdot \Delta x / w$ the gesture

Same recipe, new effects: page-flip (rotateY, origin at the edge), card stacks (translateZ ladders), cover-flow (rotateY per item). The math never changes — only the pivot does.



Zero → Master

achieved.

You can now explain every pixel of this animation: why the corner bends, why the snap feels physical, and why it never needs more than three panels.

`perspective · pivot · project · recycle`

THE END

Mohsen Madani